T.C.
TOKAT GAZİOSMANPAŞA ÜNİVERSİTESİ
BİLGİSAYAR TEKNOLOJİLERİ BÖLÜMÜ

MESLEKİ ÇÖZÜMLEME DERSİ
5. HAFTA LABORATUVAR RAPORU
(Veri Tabanı Uygulaması ve Geliştirme)

Proje / Senaryo Adı	Dijital Sinema Biletleme ve Yönetim Sistemi
Öğrenci Adı Soyadı	Gencer Yıldız
Öğrenci Numarası	254309026
GitHub Depo (Nihai Sürüm) Linki [ZORUNLU]	https://github.com/GencerYILDIZ/-Dijital-Sinema-Biletleme-ve-Yonetim-Sistemi.git

1. Projenin Amacı ve Haftalık Hedefler

Bu hafta veri tabanı projenizde hangi aşamaları tamamlamayı hedeflediğinizi kısaca açıklayınız.

Bu hafta veri tabanı projemde er diyagramında kullandığımız yapıyı kulalnarak belirlemiş olduğum veri tabanı uygulamamla birlikte veri tabanı kodlarımı yazmayı hedefliyorum.

2. Yapılan Çalışmaların Detaylı Açıklaması (ZORUNLU ALAN)

Bu hafta laboratuvarda ve projenizde tam olarak neler yaptığınızı adım adım anlatınız. Tabloların fiziksel olarak oluşturulması, ilişkilerin (Foreign Key vb.) veri tabanında tanımlanması, veri tiplerinin seçimi gibi teknik kararlarınızı gerekçeleriyle yazınız.

Bu hafta laboratuvarda CRUD kavramını işledik. Ben de web sitesi için kullanacağım veri tabanı sistemi olan PostgreSQL'i bilgisayarıma indirip kurdum. Biraz kurcalayarak yapısını ve ne yapmam gerektiğini anlamaya çalıştım. Sonrasında geçen hafta projem için hazırladığım, birincil anahtarları (PK), yabancı anahtarları (FK) ve ilişkileri gösteren ER diyagramından yararlanarak projemin veri tabanını oluşturmaya başladım.

Bu süreçte yapay zekâdan yardım alarak ilerledim. Kullandığım Django kütüphanesi sayesinde SQL dilini doğrudan yazmadan, Python ile veri tabanı yapılarımı oluşturabildim. Django, PostgreSQL ile tam uyumlu çalıştığı için oluşturduğum tablolar üzerinde PostgreSQL'de sorgulama da yapabiliyorum. Bu sırada Django'da tabloların class yapısıyla tanımlandığını ve bu işlemlerin Python ile yürütüldüğünü öğrendim.

Her tablo için bir class yazdım ve ER diyagramındaki varlıkları (Kullanıcı, Film, Tür, Salon, Koltuk, Seans, Sipariş, Bilet) bu şekilde tanımladım. Class'ın içindeki her satır, tablodaki bir sütuna karşılık geliyor. Ardından "python manage.py makemigrations" ve "python manage.py migrate" komutlarını çalıştırdım; Django bu class'lardan SQL komutlarını kendisi üretti ve tablolar PostgreSQL'de oluştu. Daha sonra pgAdmin'i açıp tabloların gerçekten oluştuğunu kontrol ettim.

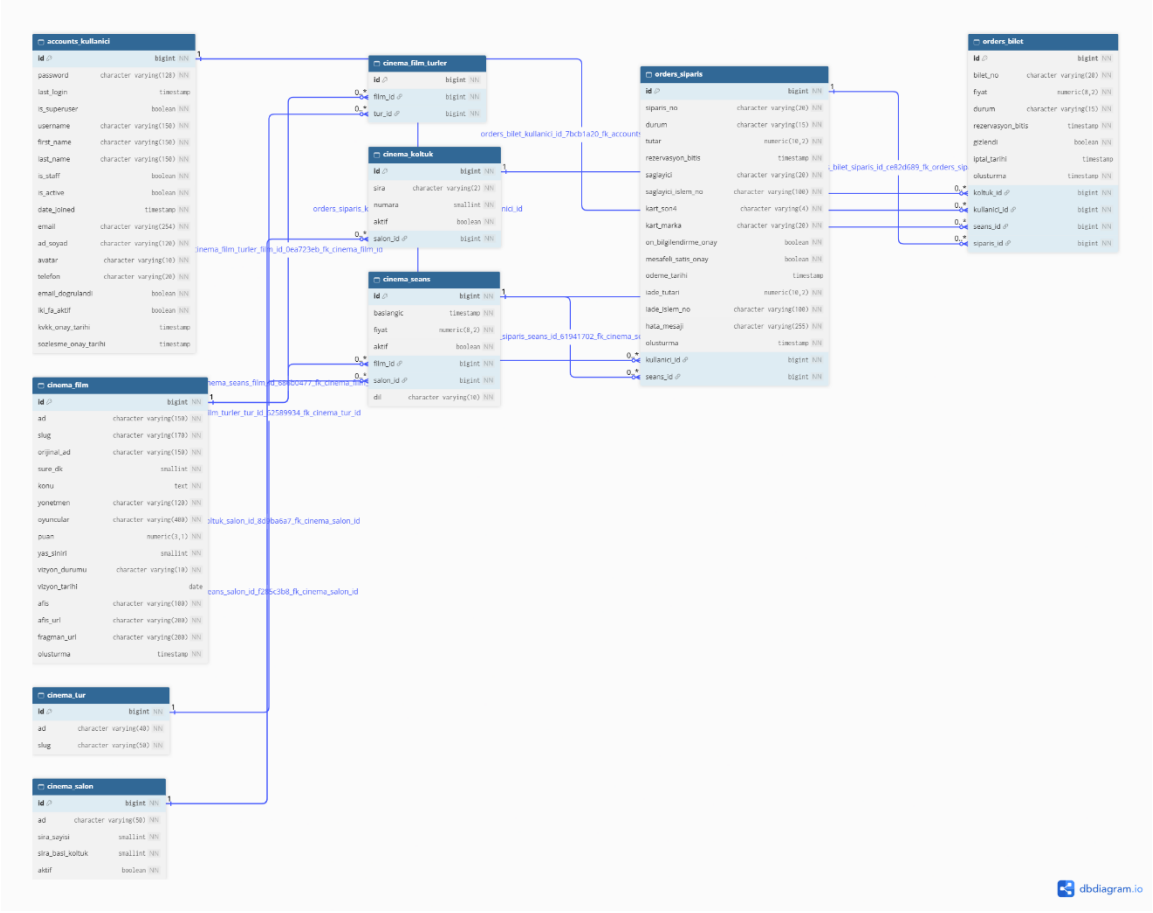
Birincil anahtar (PK) için ayrıca bir şey yazmama gerek kalmadı; Django her tabloya otomatik artan bir "id" sütunu ekledi. Tablolar arasındaki ilişkileri ise ER diyagramına bakarak yabancı anahtarlarla (FK) kurdum. Örneğin Seans tablosuna hangi filme ve hangi salona ait olduğunu gösteren film_id ve salon_id sütunlarını, Bilet tablosuna da hangi siparişe ve hangi koltuğa ait olduğunu gösteren sütunları ekledim. Bir kayıt silindiğinde ne olacağına da karar verdim: bir salon silinirse koltukları da silinsin, ancak içinde satış olan bir seans veya kullanıcı silinemesin; çünkü satılmış biletlerin sahipsiz kalmasını istemedim. Bir filmin birden fazla türü olabildiği için Film ile Tür arasındaki ilişkiyi ara bir tabloyla kurdum.

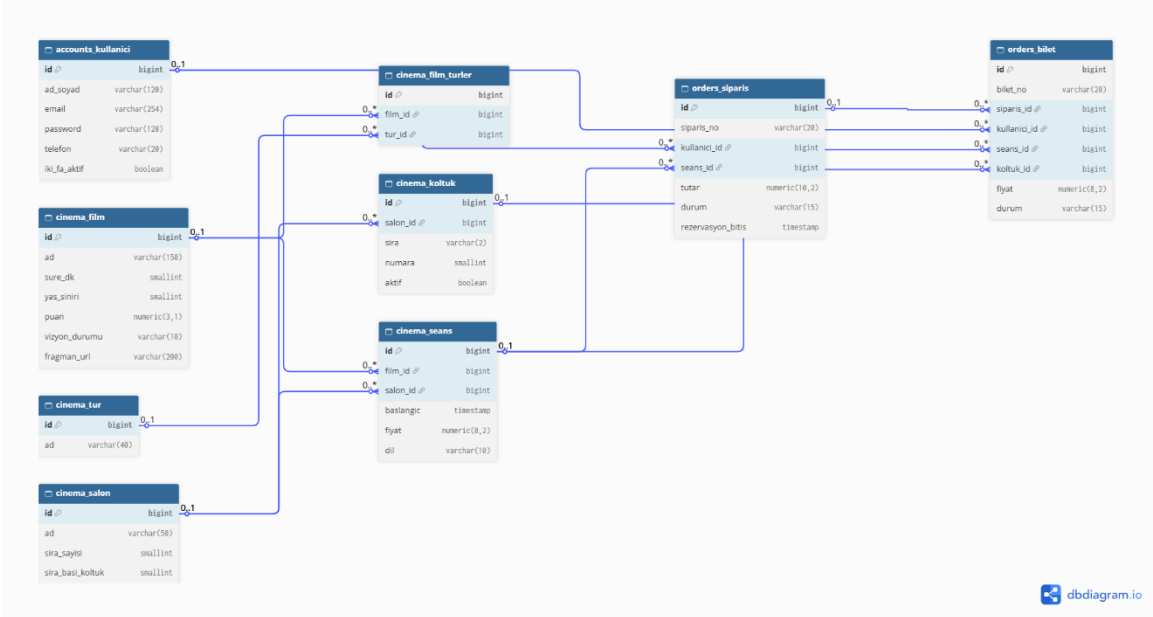
Veri tiplerini seçerken her sütunda ne tutulacağını düşündüm. Film adı ve e-posta gibi metinler için VARCHAR, film süresi ve koltuk numarası gibi küçük sayılar için SMALLINT, seansın başlangıcı için tarih ve saati birlikte tutan TIMESTAMP kullandım. Fiyat ve tutar için DECIMAL seçtim, çünkü para hesabında küsurat hatası olmaması gerekiyor. Ayrıca "aynı seansta aynı koltuk iki kez satılamaz" ve "aynı salonda aynı saatte iki seans olamaz" gibi kuralları doğrudan veri tabanına ekledim; böylece yanlış bir kayıt girilmeye çalışılsa bile veri tabanı bunu kabul etmiyor.

Laboratuvarda öğrendiğimiz CRUD işlemlerini de projemde karşılık bulduğu yerlerle eşleştirdim. Kullanıcı bilet aldığında veri tabanına yeni bir kayıt ekleniyor (Create), filmler ve seanslar listelendiğinde kayıtlar okunuyor (Read), ödeme yapıldığında biletin durumu "satıldı" olarak güncelleniyor (Update), yönetici bir seansı kaldırdığında ise kayıt siliniyor (Delete). Son olarak tablolara örnek filmler, salonlar ve seanslar ekleyip pgAdmin üzerinden kontrol ettim ve veri tabanının yedeğini aldım.

3. Veri Tabanı Şeması ve Tablo Yapıları

Oluşturduğunuz veri tabanının son halini yansıtan fiziksel şema diyagramını veya tabloların detaylı listesini (sütun adları, veri tipleri, PK/FK durumları) buraya ekleyiniz.


accounts_kullanici
id bigint NN
password character varying(128) NN
last_login timestamp
is_superuser boolean NN
username character varying(150) NN
first_name character varying(150) NN
last_name character varying(150) NN
is_staff boolean NN
is_active boolean NN
date_joined timestamp NN
email character varying(254) NN
ad_soyad character varying(120) NN
avatar character varying(10)
telefon character varying(20) NN
email_dogrulandi boolean NN
iki_fa_aktif boolean NN
kvkk_onay_tarihi timestamp
sozlesme_onay_tarihi timestamp
cinema_film_turler
id bigint NN
film_id bigint NN
tur_id bigint NN
cinema_koltuk
id bigint NN
sira character varying(2) NN
numara smallint NN
aktif boolean NN
salon_id bigint NN
cinema_seans
id bigint NN
baslangic timestamp NN
fiyat numeric(8,2) NN
aktif boolean NN
film_id bigint NN
salon_id bigint NN
dil character varying(10) NN
orders_siparis
id bigint NN
siparis_no character varying(20) NN
durum character varying(15) NN
tutar numeric(10,2) NN
rezervasyon_bitis timestamp NN
saglayici character varying(20) NN
saglayici_islem_no character varying(100) NN
kart_son4 character varying(4) NN
kart_marka character varying(20) NN
on_bilgilendirme_onay boolean NN
mesafeli_satis_onay boolean NN
odeme_tarihi timestamp
iade_tutari numeric(10,2) NN
iade_islem_no character varying(100) NN
hata_mesaji character varying(255) NN
olusturma timestamp NN
kullanici_id bigint NN
seans_id bigint NN
orders_bilet
id bigint NN
bilet_no character varying(20) NN
fiyat numeric(8,2) NN
durum character varying(15) NN
rezervasyon_bitis timestamp NN
gizlendi boolean NN
iptal_tarihi timestamp
olusturma timestamp NN
koltuk_id bigint NN
kullanici_id bigint NN
seans_id bigint NN
siparis_id bigint NN
cinema_film
id bigint NN
ad character varying(150) NN
slug character varying(170) NN
orijinal_ad character varying(150) NN
sure_dk smallint NN
konu text NN
yonetmen character varying(120) NN
oyuncular character varying(400) NN
puan numeric(3,1) NN
yas_siniri smallint NN
vizyon_durumu character varying(10) NN
vizyon_tarihi date
afis character varying(100) NN
afis_url character varying(200) NN
fragman_url character varying(200) NN
olusturma timestamp NN
cinema_tur
id bigint NN
ad character varying(40) NN
slug character varying(50) NN
cinema_salon
id bigint NN
ad character varying(50) NN
sira_sayisi smallint NN
sira_basi_koltuk smallint NN
aktif boolean NN
dbdiagram.io


accounts_kullanici
id bigint
ad_soyad varchar(120)
email varchar(254)
password varchar(128)
telefon varchar(20)
iki_fa_aktif boolean
cinema_film_turler
id bigint
film_id bigint
tur_id bigint
orders_siparis
id bigint
siparis_no varchar(20)
kullanici_id bigint
seans_id bigint
tutar numeric(10,2)
durum varchar(15)
rezervasyon_bitis timestamp
orders_bilet
id bigint
bilet_no varchar(20)
siparis_id bigint
kullanici_id bigint
seans_id bigint
koltuk_id bigint
fiyat numeric(8,2)
durum varchar(15)
cinema_koltuk
id bigint
salon_id bigint
sira varchar(2)
numara smallint
aktif boolean
cinema_film
id bigint
ad varchar(150)
sure_dk smallint
yas_siniri smallint
puan numeric(3,1)
vizyon_durumu varchar(10)
fragman_url varchar(200)
cinema_seans
id bigint
film_id bigint
salon_id bigint
baslangic timestamp
fiyat numeric(8,2)
dil varchar(10)
cinema_tur
id bigint
ad varchar(40)
cinema_salon
id bigint
ad varchar(50)
sira_sayisi smallint
sira_basi_koltuk smallint
dbdiagram.io

Tablo	Birincil Anahtar (PK)	Yabancı Anahtarlar (FK)
accounts_kullanici	id	—
cinema_film	id	—
cinema_tur	id	—
cinema_salon	id	—
cinema_film_turler	id	film_id → cinema_film, tur_id → cinema_tur
cinema_koltuk	id	salon_id → cinema_salon
cinema_seans	id	film_id → cinema_film, salon_id → cinema_salon
orders_siparis	id	kullanici_id → accounts_kullanici, seans_id → cinema_seans
orders_bilet	id	siparis_id → orders_siparis, kullanici_id → accounts_kullanici, seans_id → cinema_seans, koltuk_id → cinema_koltuk

4. Örnek SQL Sorguları / Kod Parçacıkları

Veri tabanınızı oluştururken (CREATE TABLE) veya veri manipülasyonu yaparken (INSERT, UPDATE, SELECT) kullandığınız en temel SQL sorgularından 2-3 örnek ekleyiniz.

Örnek 1: Vizyondaki filmler (SELECT ve WHERE)
Ana sayfada “Vizyonda” bölümünde gösterilen filmler:

sql

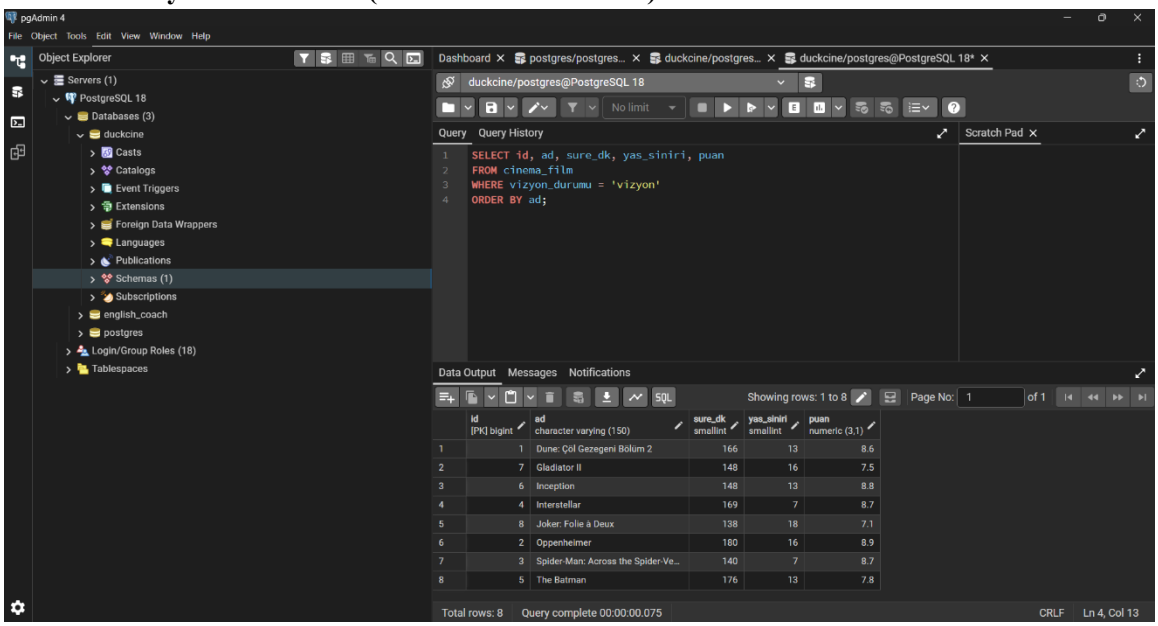
```
SELECT id, ad, sure_dk, yas_siniri, puan
FROM cinema_film
WHERE vizyon_durumu = 'vizyon'
ORDER BY ad;
```

Örnek 2: Vizyonda olmayan filmler
Henüz vizyona girmemiş (yakında) veya vizyondan kalkmış (arşiv) filmler:

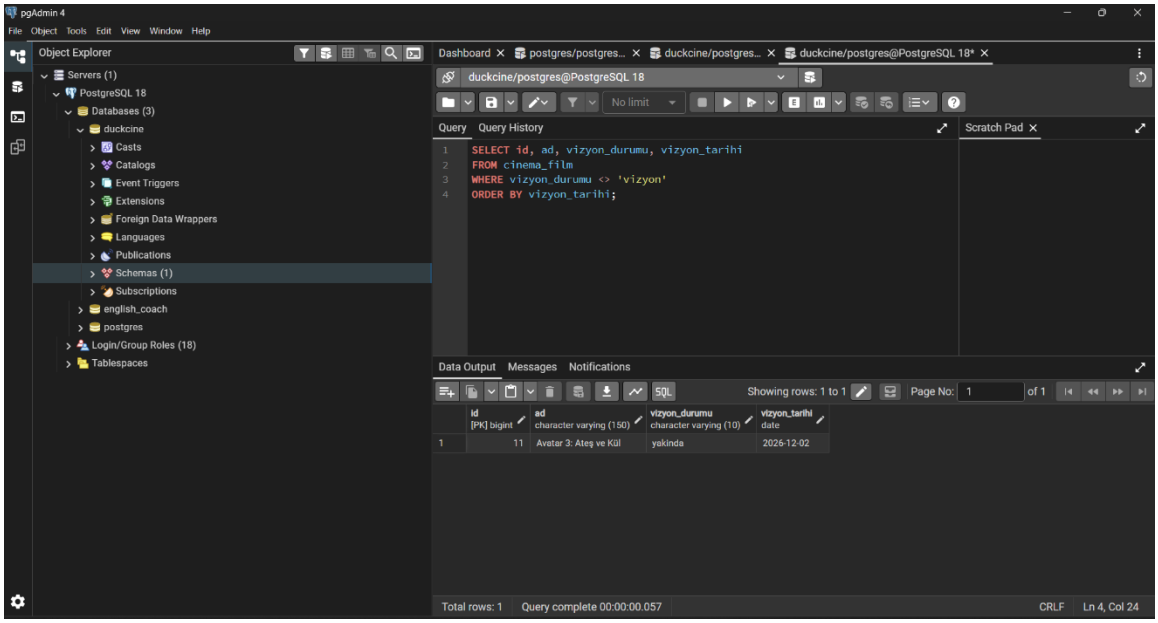
sql

```
SELECT id, ad, vizyon_durumu, vizyon_tarihi
FROM cinema_film
WHERE vizyon_durumu <> 'vizyon'
ORDER BY vizyon_tarihi;
```

Örnek 1: Vizyondaki filmler (SELECT ve WHERE):



Örnek 2: Vizyonda olmayan filmler



Örnek 3: CREATE TABLE cinema_seans

```
CREATE TABLE cinema_seans (
    id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    baslangic timestamp with time zone NOT NULL,
    fiyat numeric(8, 2) NOT NULL,
    aktif boolean NOT NULL,
    film_id bigint NOT NULL REFERENCES cinema_film (id),
    salon_id bigint NOT NULL REFERENCES cinema_salon (id),
    dil character varying(10) NOT NULL,
    CONSTRAINT salonda_ayni_saatte_tek_seans UNIQUE (salon_id, baslangic)
);
```

5. Karşılaşılan Zorluklar ve Çözümler

Tabloları birleştirirken, verileri girerken veya GitHub'a yüklerken yaşadığınız teknik problemleri ve bunları nasıl çözdüğünüzü anlatınız.

SQL diline tam hakim olmadığım için bu aşamada biraz zorlandım; ancak araştırarak ve yapay zekâdan yararlanarak ilerlediğimde bu sorunu aşabildim. Django sayesinde tabloları Python ile oluşturabildiğim için SQL'i elle yazmam gerekmedi, sadece üretilen kodları okuyup anlamaya çalıştım.

Asıl zorlandığım kısım PostgreSQL ve pgAdmin oldu. Önce pgAdmin veri tabanına bağlanamadı; sebebin PostgreSQL servisinin kapalı olması olduğunu fark edip servisi başlattım. Ardından pgAdmin hiç açılmadı; Windows'un güvenlik özelliği programın bir dosyasını engelliyordu, pgAdmin'in güncel sürümünü ayrı olarak kurunca sorun çözüldü. Bu sırada yönetici şifresini de unuttuğumu fark ettim ve verilere dokunmadan şifreyi sıfırladım.

Şema görselini hazırlarken de pgAdmin'in diyagramında tablolar arasındaki ilişki çizgileri resme çıkmadı. Tabloları Django'nun migrate komutu oluşturdu. Ben de oluşan tabloların SQL kodunu pgAdmin'in Generate SQL özelliğiyle çıkardım. Bu kodu dbdiagram.io sitesine aktararak ilişkileri net gösteren bir şema elde ettim. Son olarak şema için kullandığım SQL kodlarını proje raporuyla birlikte GitHub'a yükledim.

Öğrenci İmzası:

